UPC · ASI705 · SEMANA 4 (COMPLEMENTO SEMANA 3)

Taller paso a paso: Relaciones JPA con `@OneToMany` y `@ManyToOne`

⏱ 90 min | 🧪 Nivel: intermedio | 📦 Spring Boot · JPA · PostgreSQL

¿Qué vas a construir? Una API REST en Spring Boot con arquitectura completa en capas: `Entity → DTO → Repository → Service → Controller`. Dos entidades relacionadas: `Autor` y `Libro` (1:N).

¿Qué vas a aprender? A mapear relaciones JPA con `@OneToMany` / `@ManyToOne`, a separar el modelo de BD del contrato HTTP usando **DTOs**, a colocar la lógica de negocio en `@Service` con `@Transactional`, y a hacer consultas derivadas sin escribir SQL.

Requisitos previos: haber completado el taller de S3 (entidades y repositorios básicos).

---

1 Preparar el proyecto

Partimos del mismo proyecto que usaste en Semana 3. Si lo perdiste, crea uno nuevo en Spring Initializr con estas dependencias:

- Spring Web
- Spring Boot DevTools
- Spring Data JPA
- PostgreSQL Driver
- Lombok

Además, agrega manualmente en `pom.xml` la dependencia de Swagger (no aparece en Initializr):

pom.xml

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.5.0</version>
</dependency>
```

APPLICATION.PROPERTIES

Crea una base nueva llamada `biblioteca_db` en PostgreSQL y configúrala:

src/main/resources/application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/biblioteca_db
spring.datasource.username=postgres
spring.datasource.password=tu_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

💡 `ddl-auto=update` hace que Hibernate cree y modifique las tablas automáticamente a partir de tus entidades. En producción se usa `validate` o migraciones (Flyway), pero para clase `update` es cómodo.

---

2 Modelo mental — ¿qué es una relación `1:N` ?

En la vida real:

En la base de datos se traduce a dos tablas, donde la tabla "hija" guarda la referencia al padre por medio de una **llave foránea (FK)**:

JPA mapea esto con dos anotaciones clave:

- `@ManyToOne` del lado **Libro** — "muchos libros pertenecen a un autor"
- `@OneToMany` del lado **Autor** — "un autor tiene muchos libros"

💡 La **FK siempre vive en el lado** `@ManyToOne` (el lado "N"). El lado `@OneToMany` solo refleja la relación, no crea columnas nuevas.

---

3 Entidad `Autor`

src/main/java/com/tuempresa/biblioteca/entities/Autor.java

```
package com.tuempresa.biblioteca.entities;

import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "autor")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class Autor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(length = 60)
    private String nacionalidad;

    @OneToMany(
        mappedBy = "autor",        // <-- el campo "autor" de Libro es el dueño
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<Libro> libros = new ArrayList<>();
}
```

¿POR QUÉ CADA ANOTACIÓN?

Anotación	Para qué sirve
`mappedBy = "autor"`	Indica que la FK está en el campo `autor` de `Libro`. Evita crear una tabla extra.
`cascade = ALL`	Si guardas un autor con libros nuevos, los libros se guardan solos.
`orphanRemoval = true`	Si quitas un libro de la lista, se borra de la BD (no se queda huérfano).

💡 No necesitamos `@JsonManagedReference` / `@JsonBackReference` porque nunca devolveremos la entidad por la API: siempre retornamos un DTO (siguiente paso). Eso rompe el ciclo sin anotaciones extra.

---

4 Entidad `Libro` (el lado dueño)

`src/main/java/com/tuempresa/biblioteca/entities/Libro.java`

```java
package com.tuempresa.biblioteca.entities;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "libro")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String titulo;

    @Column(nullable = false)
    private Integer anio;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "autor_id", nullable = false)  // <-- columna FK real
    private Autor autor;
}
```

💡 `fetch = LAZY` evita traer al autor (y toda su lista de libros) cada vez que cargas un Libro. Carga el autor solo si lo pides explícitamente.

¿CUÁNDO USAR `@COLUMN` Y CUÁNDO `@COLUMN(NAME = "...")`?

Duda muy común: "profe, ¿por qué `nombre` tiene `@Column` y `libros` no?" o "¿por qué aquí va `name = "..."` y aquí no?". Regla simple:

Situación	Qué escribir	Por qué
El campo existe y quieres los defaults (VARCHAR(255), nullable, columna = mismo nombre)	*Nada, omite* `@Column`	JPA ya aplica todo eso por defecto. Agregar `@Column` sin parámetros es ruido.
Quieres restringir el campo (no nulo, longitud, unique)	`@Column(nullable = false, length = 100)`	Solo especificas los parámetros que cambian respecto al default.
El campo es camelCase en Java pero quieres snake_case en la BD	`@Column(name = "fecha_publicacion")` sobre `private LocalDate fechaPublicacion`	Hibernate no convierte camelCase automáticamente (salvo que configures un naming strategy). Por eso debes nombrar la columna explícitamente.
El nombre del campo coincide con una palabra reservada de SQL (`user` , `order` , `group`)	`@Column(name = "usuario")`	Evita errores de sintaxis SQL al generar las queries.
El campo es una relación (`@OneToMany` , `@ManyToOne`)	No usar `@Column` ahí	Las relaciones se mapean con `@JoinColumn` (para FK) o con `mappedBy` (para el lado inverso). `@Column` es solo para columnas escalares.

💡 En nuestro taller: `id` no lleva `@Column` porque `@Id + @GeneratedValue` ya lo configuran. `nombre` lleva `@Column(nullable = false, length = 100)` porque queremos esas restricciones. `nacionalidad` lleva `@Column(length = 60)` para acotar el tamaño. `libros` no lleva `@Column` porque es una relación (`@OneToMany`).

LO QUE HIBERNATE GENERA EN POSTGRESQL

```sql
CREATE TABLE autor (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    nacionalidad VARCHAR(255)
);

CREATE TABLE libro (
    id BIGSERIAL PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    anio INTEGER,
    autor_id BIGINT NOT NULL REFERENCES autor(id)
);
```

---

5 DTOs (Data Transfer Object)

La **entidad** modela la tabla de la base de datos. El **DTO** modela el contrato entre tu API y quien la consume (Postman, una app web, otro microservicio). Son cosas distintas y deben estar en clases distintas.

¿POR QUÉ SEPARAR ENTIDAD Y DTO?

- La entidad puede tener relaciones bidireccionales (Autor↔Libro) que, si las serializas directo, generan un loop infinito al convertir a JSON.
- La entidad puede tener campos sensibles o internos que no deberías exponer (ej. `password`, `deletedAt`).
- El contrato de la API debe ser estable: si mañana cambias una columna de la BD, no deberías romper a los clientes.
- Rosa (y el sílabo del curso) lo exigen explícitamente en la PC1.

💡 Con DTO, el flujo es: `Entity` vive en la capa de persistencia, el `DTO` viaja por HTTP. El mapeo `Entity ↔ DTO` lo hace el Service (paso 7).

DTO DEL AUTOR

```
.../dtos/AutorDTO.java
```

```java
package com.tuempresa.biblioteca.dtos;

import com.tuempresa.biblioteca.entities.Autor;
import lombok.*;
import java.util.List;
import java.util.stream.Collectors;

@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class AutorDTO {

    private Long id;
    private String nombre;
    private String nacionalidad;
    private List<LibroDTO> libros;

    // Entity -> DTO
    public static AutorDTO fromEntity(Autor a) {
        List<LibroDTO> libros = a.getLibros() == null ? List.of()
            : a.getLibros().stream()
                .map(LibroDTO::fromEntity)
                .collect(Collectors.toList());
        return new AutorDTO(a.getId(), a.getNombre(), a.getNacionalidad(), libros);
    }

    // DTO -> Entity (sin libros: los libros se crean por separado vía LibroService)
    public Autor toEntity() {
        Autor a = new Autor();
        a.setId(this.id);
        a.setNombre(this.nombre);
        a.setNacionalidad(this.nacionalidad);
        return a;
    }
}
```

DTO DEL LIBRO

`.../dtos/LibroDTO.java`

```java
package com.tuempresa.biblioteca.dtos;

import com.tuempresa.biblioteca.entities.Libro;
import lombok.*;

@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class LibroDTO {

    private Long id;
    private String titulo;
    private Integer anio;
    private Long autorId;        // <-- solo el ID del autor, NO la entidad completa

    public static LibroDTO fromEntity(Libro l) {
        Long aid = l.getAutor() == null ? null : l.getAutor().getId();
        return new LibroDTO(l.getId(), l.getTitulo(), l.getAnio(), aid);
    }

    public Libro toEntity() {
        Libro l = new Libro();
        l.setId(this.id);
        l.setTitulo(this.titulo);
        l.setAnio(this.anio);
        // autor lo asigna el Service buscando por autorId
        return l;
    }
}
```

💡 Observa que `LibroDTO` expone `autorId` (un `Long`), no una referencia a `Autor`. Eso rompe el ciclo Autor→Libro→Autor y hace el JSON plano y legible.

---

6 Repositorios

Dos repositorios muy simples. Spring Data JPA genera las implementaciones solo.

`.../repositories/AutorRepository.java`

```java
package com.tuempresa.biblioteca.repositories;

import com.tuempresa.biblioteca.entities.Autor;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AutorRepository extends JpaRepository<Autor, Long> { }
```

.../repositories/LibroRepository.java

```java
package com.tuempresa.biblioteca.repositories;

import com.tuempresa.biblioteca.entities.Libro;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface LibroRepository extends JpaRepository<Libro, Long> {

    // Consulta derivada: Spring la construye a partir del nombre del método.
    List<Libro> findByAutorId(Long autorId);

    // Otra consulta derivada: buscar por título (like, ignorando mayúsculas).
    List<Libro> findByTituloContainingIgnoreCase(String fragmento);
}
```

💡 Los nombres `findByAutorId` y `findByTituloContainingIgnoreCase` no son casualidad: Spring Data los parsea y genera el SQL. La convención es `findBy` + nombre del atributo + operadores (`Containing` , `IgnoreCase` , `GreaterThan` , etc.).

---

7 Services (lógica de negocio + mapeo)

El **Service** es la capa que contiene la lógica del caso de uso. Es quien:

- Llama al repositorio para leer o escribir.
- Convierte entidades a DTOs y viceversa.
- Valida reglas de negocio (por ejemplo, "no crear libro si el autor no existe").
- Se anota con `@Service` para que Spring lo registre.

El **Controller** delega todo al Service y nunca toca el Repository directamente.

AUTORSERVICE

.../services/AutorService.java

```java
package com.tuempresa.biblioteca.services;

import com.tuempresa.biblioteca.dtos.AutorDTO;
import com.tuempresa.biblioteca.entities.Autor;
import com.tuempresa.biblioteca.repositories.AutorRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class AutorService {

    private final AutorRepository autorRepo;

    public AutorService(AutorRepository autorRepo) {
        this.autorRepo = autorRepo;
    }

    @Transactional(readOnly = true)
    public List<AutorDTO> listar() {
        return autorRepo.findAll().stream()
            .map(AutorDTO::fromEntity)
            .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public AutorDTO obtenerPorId(Long id) {
        Autor a = autorRepo.findById(id)
            .orElseThrow(() -> new RuntimeException("Autor con id " + id + " no existe"
        return AutorDTO.fromEntity(a);
    }

    @Transactional
    public AutorDTO crear(AutorDTO dto) {
        Autor nuevo = dto.toEntity();
        Autor guardado = autorRepo.save(nuevo);
        return AutorDTO.fromEntity(guardado);
    }
}
```

LIBROSERVICE

`.../services/LibroService.java`

```java
package com.tuempresa.biblioteca.services;

import com.tuempresa.biblioteca.dtos.LibroDTO;
import com.tuempresa.biblioteca.entities.Autor;
import com.tuempresa.biblioteca.entities.Libro;
import com.tuempresa.biblioteca.repositories.AutorRepository;
import com.tuempresa.biblioteca.repositories.LibroRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class LibroService {

    private final LibroRepository libroRepo;
    private final AutorRepository autorRepo;

    public LibroService(LibroRepository libroRepo, AutorRepository autorRepo) {
        this.libroRepo = libroRepo;
        this.autorRepo = autorRepo;
    }

    @Transactional(readOnly = true)
    public List<LibroDTO> listar() {
        return libroRepo.findAll().stream()
            .map(LibroDTO::fromEntity)
            .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public List<LibroDTO> porAutor(Long autorId) {
        return libroRepo.findByAutorId(autorId).stream()
            .map(LibroDTO::fromEntity)
            .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public List<LibroDTO> buscarPorTitulo(String q) {
        return libroRepo.findByTituloContainingIgnoreCase(q).stream()
            .map(LibroDTO::fromEntity)
            .collect(Collectors.toList());
    }

    @Transactional
    public LibroDTO crearParaAutor(Long autorId, LibroDTO dto) {
        Autor autor = autorRepo.findById(autorId)
            .orElseThrow(() -> new RuntimeException("Autor con id " + autorId + " no ex

        Libro libro = dto.toEntity();
```

```
        libro.setAutor(autor);
        Libro guardado = libroRepo.save(libro);
        return LibroDTO.fromEntity(guardado);
    }
}
```

💡 `@Transactional(readOnly = true)` en las consultas le dice a Hibernate que no abra transacción de escritura, lo que es más rápido. En métodos que modifican la BD usa `@Transactional` sin `readOnly`.

---

8 Controllers (delegando al Service, devolviendo DTOs)

Los controllers ahora son delgados: reciben la request, delegan al service y devuelven DTOs. Nunca tocan el Repository ni la Entity directamente.

`.../controllers/AutorController.java`

```java
package com.tuempresa.biblioteca.controllers;

import com.tuempresa.biblioteca.dtos.AutorDTO;
import com.tuempresa.biblioteca.services.AutorService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/autores")
public class AutorController {

    private final AutorService autorService;

    public AutorController(AutorService autorService) {
        this.autorService = autorService;
    }

    @GetMapping
    public List<AutorDTO> listar() {
        return autorService.listar();
    }

    @GetMapping("/{id}")
    public AutorDTO obtenerPorId(@PathVariable Long id) {
        return autorService.obtenerPorId(id);
    }

    @PostMapping
    public AutorDTO crear(@RequestBody AutorDTO dto) {
        return autorService.crear(dto);
    }
}
```

.../controllers/LibroController.java

```java
package com.tuempresa.biblioteca.controllers;

import com.tuempresa.biblioteca.dtos.LibroDTO;
import com.tuempresa.biblioteca.services.LibroService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/libros")
public class LibroController {

    private final LibroService libroService;

    public LibroController(LibroService libroService) {
        this.libroService = libroService;
    }

    @GetMapping
    public List<LibroDTO> listar() {
        return libroService.listar();
    }

    @GetMapping("/por-autor/{autorId}")
    public List<LibroDTO> porAutor(@PathVariable Long autorId) {
        return libroService.porAutor(autorId);
    }

    @GetMapping("/buscar")
    public List<LibroDTO> buscar(@RequestParam String q) {
        return libroService.buscarPorTitulo(q);
    }

    @PostMapping("/autor/{autorId}")
    public LibroDTO crearParaAutor(
            @PathVariable Long autorId,
            @RequestBody LibroDTO dto) {
        return libroService.crearParaAutor(autorId, dto);
    }
}
```

💡 Compara: el controller ahora tiene 3-4 líneas por método. Toda la lógica vive en el Service. Eso es lo que se llama "controlador delgado, servicio gordo".

9 Documentar y probar con Swagger

¿QUÉ ES SWAGGER?

Swagger (hoy parte del estándar **OpenAPI**) es una herramienta que lee las anotaciones de tus controllers (`@RestController` , `@GetMapping` , etc.) y genera automáticamente:

- Una **especificación JSON** de toda tu API (rutas, métodos, bodies, respuestas).
- Una **interfaz web** (Swagger UI) para probar los endpoints sin instalar Postman.

La dependencia que agregamos en el Paso 1 (`springdoc-openapi-starter-webmvc-ui`) se encarga de todo. No necesitas configurar nada más.

ARRANCAR LA APP

```
mvn spring-boot:run
```

URLS IMPORTANTES

URL	Qué sirve
`http://localhost:8080/swagger-ui.html`	Interfaz gráfica para probar la API
`http://localhost:8080/v3/api-docs`	Especificación JSON (OpenAPI)

CÓMO USAR SWAGGER UI

1. Abre `http://localhost:8080/swagger-ui.html` en el navegador.
2. Verás las secciones **autor-controller** y **libro-controller** con todos los endpoints agrupados.
3. Haz clic en cualquier endpoint (por ejemplo `POST /api/autores`) → botón **"Try it out"**.
4. Edita el JSON del body y presiona **"Execute"**. Swagger mostrará la URL real, el cURL equivalente, y la respuesta del servidor.

💡 Swagger es la manera más rápida de que un compañero (o el profesor) pruebe tu API sin tener que escribir código ni configurar clientes. Es documentación viva: cambia un endpoint y Swagger se actualiza solo.

OPCIONAL: ENRIQUECER LA DOCUMENTACIÓN

Puedes anotar tus controllers para que Swagger muestre descripciones más claras:

```java
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@RequestMapping("/api/autores")
@Tag(name = "Autores", description = "Gestión de autores y sus libros")
public class AutorController {

    @Operation(summary = "Lista todos los autores con sus libros")
    @GetMapping
    public List<Autor> listar() { ... }

    @Operation(summary = "Crea un autor (opcionalmente con libros anidados)")
    @PostMapping
    public Autor crear(@RequestBody Autor autor) { ... }
}
```

---

10 Pruebas concretas (Swagger o Postman)

Arranca la app con `mvn spring-boot:run`. Base URL: `http://localhost:8080`. Puedes ejecutar esta secuencia en Swagger UI o en Postman — los resultados deben ser idénticos.

Paso	Método	URL	Body
1. Crear autor	POST	`/api/autores`	`{"nombre":"Gabriel García Márquez","nacionalidad":"Colombia"}`
2. Agregar libro al autor 1	POST	`/api/libros/autor/1`	`{"titulo":"Cien años de soledad","anio":1967}`
3. Agregar otro libro al autor 1	POST	`/api/libros/autor/1`	`{"titulo":"El amor en los tiempos del cólera","anio":1985}`
4. Obtener autor 1 con sus libros	GET	`/api/autores/1`	—
5. Listar libros del autor 1	GET	`/api/libros/por-autor/1`	—
6. Buscar por título	GET	`/api/libros/buscar?q=amor`	—

11 Verificar en la base de datos

Conéctate a PostgreSQL con `psql`, pgAdmin o DBeaver y corre:

```
SELECT a.nombre, l.titulo, l.anio
FROM autor a
JOIN libro l ON l.autor_id = a.id
ORDER BY a.nombre, l.anio;
```

Deberías ver una fila por cada libro, con el nombre del autor al lado. Si ves esto, la relación `@OneToMany` está funcionando correctamente.

12 Errores comunes y cómo resolverlos

Lista de cosas con las que tus compañeros ya chocaron. Antes de gritar al profe, revisa esto.

1 · MAVEN NO ENCUENTRA LA DEPENDENCIA QUE ACABO DE AGREGAR

Síntoma: agregas `springdoc-openapi` o `lombok` al `pom.xml`, pero IntelliJ sigue diciendo `Cannot resolve symbol Operation` o `Cannot find Getter`.

Causa: IntelliJ no recarga el `pom.xml` automáticamente.

Solución (elige una):

- En la esquina superior derecha aparece un pequeño ícono flotante de Maven (una M con flecha circular) cada vez que cambias el pom. Haz clic ahí y dice *"Load Maven Changes"*.
- Atajo de teclado: `Cmd + Shift + O` (Mac) o `Ctrl + Shift + O` (Windows/Linux).
- Vía menú: click derecho sobre `pom.xml` → **Maven** → **Reload project**.
- Desde el panel Maven (derecha): botón de *Reload All Maven Projects* (ícono de flechas circulares).

💡 Regla de oro: **cada vez que tocas `pom.xml`**, recarga Maven. Si no, los cambios simplemente no existen para IntelliJ.

2 · "FAILED TO CONFIGURE A DATASOURCE" AL ARRANCAR

Síntoma: la app no arranca y el log dice *"Failed to configure a DataSource: 'url' attribute is not specified"*.

Causa: falta `application.properties` o está mal escrito.

Solución: revisa que `src/main/resources/application.properties` tenga `spring.datasource.url`, `spring.datasource.username` y `spring.datasource.password` con los valores correctos de tu PostgreSQL local.

3 · "DATABASE 'BIBLIOTECA_DB' DOES NOT EXIST"

Síntoma: el log dice exactamente eso al arrancar.

Causa: Spring Boot crea las **tablas** automáticamente con `ddl-auto=update`, pero la **base de datos** la tienes que crear tú a mano.

Solución: abre pgAdmin o `psql` y ejecuta:

```
CREATE DATABASE biblioteca_db;
```

4 · "PASSWORD AUTHENTICATION FAILED FOR USER 'POSTGRES'"

Síntoma: PostgreSQL rechaza la conexión.

Solución: revisa que `spring.datasource.password` coincida con la contraseña de tu instalación de PostgreSQL. Si la olvidaste, la puedes cambiar con `ALTER USER postgres WITH PASSWORD 'nuevaclave';` desde pgAdmin.

5 · LOMBOK NO GENERA LOS GETTERS / SETTERS

Síntoma: `@Getter @Setter` están en la clase pero IntelliJ no reconoce `getNombre()`.

Solución (dos pasos obligatorios):

1. Instalar el **plugin Lombok** en IntelliJ (*Settings → Plugins → Marketplace → buscar "Lombok"*).
2. Habilitar *Annotation Processing*: *Settings → Build, Execution, Deployment → Compiler → Annotation Processors → marcar "Enable annotation processing"*.

6 · "NO PROPERTY X FOUND FOR TYPE Y" AL INICIAR LA APP

Síntoma: un `UnsatisfiedDependencyException` que termina en *"No property titulos found for type Libro"*.

Causa: escribiste un *derived query* mal (ej. `findByTitulos` pero el campo se llama `titulo`).

Solución: los nombres de método tienen que coincidir **exactamente** con los campos de la entidad (respetando mayúsculas).

7 · "LAZYINITIALIZATIONEXCEPTION" AL DEVOLVER ENTIDADES

Síntoma: al hacer `GET /api/autores`, el log truena con *"could not initialize proxy - no Session"*.

Causa: Hibernate intentó cargar los libros (LAZY) fuera de la transacción.

Solución: tu servicio debería devolver **DTOs** (como en el Paso 7), no entidades. Si devuelves DTOs, nunca chocas con este error porque el mapeo `fromEntity` se ejecuta dentro de la transacción del service.

8 · SWAGGER UI MUESTRA "NO OPERATIONS DEFINED"

Síntoma: abres `/swagger-ui.html` y dice que no hay endpoints.

Causa habitual: la dependencia de springdoc no está en `pom.xml`, o Maven no se recargó (ver error #1).

Solución: confirma en `pom.xml` que tienes `springdoc-openapi-starter-webmvc-ui` versión 2.5.0 o superior, y recarga Maven.

9 · "RELATION 'AUTOR' DOES NOT EXIST" CUANDO INTENTAS UN SELECT

Síntoma: las tablas no se ven en pgAdmin aunque la app arrancó.

Causa: Hibernate crea las tablas **recién cuando inserta el primer registro**. Si solo arrancaste la app sin hacer POST, las tablas aún no existen.

Solución: haz un `POST /api/autores` desde Postman o Swagger. Después refresca pgAdmin y verás las tablas.

10 · EL PUERTO 8080 YA ESTÁ EN USO

Síntoma: *"Web server failed to start. Port 8080 was already in use"*.

Solución rápida: agrega esta línea en `application.properties`:

```
server.port=8081
```

Y accede a `http://localhost:8081`.

CHECKLIST RÁPIDO ANTES DE LEVANTAR LA MANO

- ¿Recargaste Maven después de tocar `pom.xml` ?
- ¿PostgreSQL está corriendo?
- ¿Creaste la base de datos a mano?
- ¿Los nombres de los *derived queries* coinciden con los campos de la entidad?
- ¿Tus servicios devuelven DTOs, no entidades?
- ¿Lombok está habilitado (plugin + annotation processing)?

✅ Lo que te llevas

- Construiste una arquitectura completa en capas: `Entity → DTO → Repository → Service → Controller` .
- Sabes dónde va la FK en una relación 1:N (siempre en el lado `@ManyToOne`).
- Entiendes `mappedBy` , `cascade` y `orphanRemoval` .
- Rompiste el ciclo entidad↔entidad usando DTOs con `fromEntity` / `toEntity` , sin `@JsonManagedReference` .
- Usaste `@Service` y `@Transactional` (con `readOnly` cuando toca) para la lógica de negocio.
- Creaste consultas derivadas (`findByAutorId` , `findByTituloContainingIgnoreCase`) sin escribir SQL.

EJERCICIOS EXTRA (SI SOBRA TIEMPO)

1. Agrega el endpoint `DELETE /api/autores/{id}` y observa cómo `cascade` borra también sus libros.
2. Cambia `orphanRemoval` a `false` , quita un libro de la lista y vuelve a guardar el autor. ¿Qué pasa? ¿Por qué?
3. Agrega una consulta derivada: `findByAnioGreaterThanEqual(Integer anio)` y pruébala.